Programación Orientada a Objetos con Python

Programación para Ciencia de Datos - Entregable I

Alfonso Andrés

Índice de Contenidos

1 Enunciado . . . 5

2 Tareas a Realizar . . . 6
- 2.1 Tarea 1 . . . 6
- 2.2 Tarea 2 . . . 6
- 2.3 Tarea 3 . . . 6
- 2.4 Tarea 4 . . . 6

3 Desarrollo del Proyecto . . . 7
- 3.1 Preparación del Entorno de Trabajo . . . 7
  - 3.1.1 Creación del Repositorio Remoto y Local . . . 7
  - 3.1.2 Configuración del Archivo README.md . . . 7
  - 3.1.3 Configuración del Archivo .gitignore . . . 8
  - 3.1.4 Creación de la Estructura de Carpetas y Plantillas de Documentación 8
  - 3.1.5 Creación del Tag de Pre - Configuración del Proyecto . . . 8
  - 3.1.6 Subida de los Archivos de Documentación al Repositorio Remoto . . 9
  - 3.1.7 Creación de la Rama de Desarrollo . . . 9
- 3.2 Diseño del Diagrama de Clases, de Caso de Uso y de Secuencia en UML . 10
  - 3.2.1 Diagrama de Clases UML . . . 10
  - 3.2.2 Diagrama de Casos de Uso UML . . . 12
  - 3.2.3 Diagrama de Secuencia UML . . . 13
  - 3.2.4 Publicación de los Diagramas UML en el Repositorio Remoto . . . 14
  - 3.2.5 Creación del Tag de Diagramas UML . . . 14
- 3.3 Implementación en Python del Sistema de Gestión de la Flota Espacial . . 15
  - 3.3.1 Implementación del Código Fuente . . . 15
  - 3.3.2 Ejecución del Código Fuente . . . 16
  - 3.3.3 Creación del Tag de Implementación del Código Fuente . . . 17
- 3.4 Realización de Pruebas del Código Fuente con `pytest` . . . 18
  - 3.4.1 Implementación de Tests con `pytest` . . . 18
  - 3.4.2 Ejecución de los Tests con `pytest` . . . 18
  - 3.4.3 Creación del Tag de Pruebas Unitarias . . . 19
- 3.5 Cambios Finales y Cierre del Proyecto . . . 20

Alfonso Andrés

1 Enunciado

Nos contratan en el *Imperio Galáctico* para hacer un software que gestione el mantenimiento mecánico de la flota espacial. Esta, está compuesta por diferentes tipos de naves que se muestran a continuación. Cada nave se identifica con un **nombre *(texto)*** y lleva asociado un **catálogo con los nombres de las piezas de repuesto *(lista de texto)***. Además, los diferentes tipos de nave tienen unas propiedades específicas, que se muestran a continuación:

Tipo de Nave	**Propiedades**
Estación Espacial	Tripulación (número) Pasaje (número) Ubicación (Endor, Cúmulo Raimos, Nebulosa Kaliida)
Nave estelar	Tripulación (número) Pasaje (número) Clase (Ejecutor, Eclipse, Soberano)
Caza estelar	Dotación (número)

Table 1: Propiedades de los Diferentes Tipos de Naves Imperiales.

Todas las unidades de combate imperiales (esto es, no sólo las naves, sino también vehículos terrestres y demás) tienen un **identificador de combate *(texto)***, y una **clave para transmisión cifrada *(número)***.

Los distintos repuestos disponibles se encuentran localizados en almacenes, que están identificados por un **nombre *(texto)*** y ubicados en una **localización *(texto)***. Cada almacén dispone de un **catálogo de piezas de repuesto *(lista de repuestos)***, cada una de las cuales incluye un **nombre de repuesto *(texto)***, un **proveedor *(texto)***, la **cantidad de repuestos disponibles *(número, atributo privado)***, así como su **precio *(número)***.

Los usuarios del sistema informático *MiImperio* serán tanto los **comandantes de las naves** (para consultar y adquirir repuestos) como los **operarios de almacén** (para mantener la lista de repuestos y los stocks, en los diferentes almacenes).

Alfonso Andrés

2 Tareas a Realizar

2.1 Tarea 1

Diseña el diagrama de clases, de caso de uso y de secuencia en UML con la herramienta `Umlet`, vista en clase. En este sentido, se debe explotar en la medida de lo posible todas las técnicas POO (herencia simple y/o múltiple, clases abstractas, enumeraciones, visibilidad, etc.). **(3 puntos)**

2.2 Tarea 2

Implementa en Python el sistema de gestión de mantenimiento de la flota. Se ha de incluir el código básico de prueba que muestre mediante impresión en pantalla la instanciación correcta de las diferentes clases del sistema.

2.3 Tarea 3

El código debe de incluir una gestión de excepciones que desencadene, capture y trate de forma adecuada diferentes errores que puedan ocurrir durante la ejecución del sistema, así como un conjunto de tests unitarios definidos con pytest. **(4,5 puntos)**

2.4 Tarea 4

Durante todo el desarrollo del proyecto deberá usarse git como sistema de control de versiones. **(2,5 puntos)** Deberán realizarse las acciones siguientes:

a. Definición dos ramas dentro del repositorio **(1 punto)**:

- **i.** Una rama `main` (principal) que contendrá el proyecto final.
- **ii.** Una rama llamada **development** en donde se realizarán los `commits` durante la fase de desarrollo del proyecto.

b. Definición de dos tags dentro de la rama `development` (se deja a criterio del estudiante decidir en qué momento del proyecto realizarlos). **(0.5 puntos)**

c. Creación de un repositorio público en GitHub, titulado `pcd_entregable1_<nombresdelosmiembrosdelgrupo>`. **(1 punto)**

Alfredo Andrés

3 Desarrollo del Proyecto

3.1 Preparación del Entorno de Trabajo

3.1.1 Creación del Repositorio Remoto y Local

Vamos a comenzar con el desarrollo del proyecto creando el entorno de trabajo adecuado para el desarrollo del mismo. En primer lugar, empezaremos preparando el entorno de la nube creando repositorio público en GitHub, titulado `pcd_entregable1_alfonsoandres`, el cual podrás visitar haciendo clic aquí. Posteriormente, vamos a preparar el entorno local de desarrollo, ejecutando los siguientes comandos en la terminal:

```
# Inicializamos el repositorio local
git init

# Configuramos el usuario de git
git config --global user. email "alfonsoandres.gimenez@edu.upct.es"
git config --global user. name "alfonsoandressUPCT"

# Aseguramos que la rama principal se llame main
git branch -M main

# Agregamos el repositorio remoto
git remote add origin https://github.com/alfonsoandressUPCT/pcd_entregable1_alfonso.git

# Traemos los cambios del repositorio remoto
git pull origin main

# Realizamos el primer commit con el README.md primitivo
git add .
git commit -m "Añadiendo archivos iniciales del entregable 1"
git push -u origin main
```

3.1.2 Configuración del Archivo README.md

Una vez fusionado el repositorio remoto con el local, ya podemos trabajar con normalidad en el desarrollo del proyecto. Y lo primero que haremos será establecer un README.md profesional que especifique el proyecto, su estructura y su uso. Para ello, se ha realizado un commit para subir el README.md al repositorio remoto:

```
git add README.md
git commit -m "Modificando el README inicial del proyecto"
git push -u origin main
```

3.1.3 Configuración del Archivo .gitignore

A continuación, hemos añadido el archivo `.gitignore` para evitar subir al repositorio remoto archivos innecesarios. Como subimos por error archivos de tipo `.DS_Store` los eliminamos también. Para ello, hemos ejecutado los siguientes comandos:

```
# Eliminamos los archivos .DS_Store del repositorio
git rm --cached . DS_Store

# Realizamos un commit para eliminar los archivos .DS_Store del rastreo y añadir el .gitignore
git add .gitignore
git commit -m "Añadir .gitignore y eliminar .DS_Store del rastreo"
```

3.1.4 Creación de la Estructura de Carpetas y Plantillas de Documentación

Ahora, creamos la estructura de carpetas del proyecto, la cual se muestra a continuación:

```
pcd_entregable1_alfonso/
|-- README.md            # Archivo de documentación principal del proyecto
|-- .gitignore           # Archivo para ignorar archivos innecesarios en el repositorio
|-- docs                 # Documentación del proyecto
|   |-- data             # Datos utilizados en la documentación
|   |-- images           # Imágenes utilizadas en la documentación
|   |-- interface        # Contenido en bruto de la documentación
|   `-- uml              # Diagramas UML del proyecto
|-- src                  # Código fuente del proyecto
`-- tests                # Tests unitarios del proyecto
```

Y, posteriormente, realizamos un commit para subir la estructura de carpetas al repositorio remoto:

```
git add .
git commit -m "Definir estructura de carpetas y archivos base del proyecto"
git push origin main
```

Seguimos añadiendo los archivos plantilla de la documentación en sus respectivas carpetas, y realizando commits para subirlos al repositorio remoto de la siguiente forma:

```
git add .
git commit -m "Terminando la pre-configuración pre-documentación para preparar el entorno para la creación y
      desarrollo del proyecto"
git push -u origin main
```

3.1.5 Creación del Tag de Pre - Configuración del Proyecto

Una vez que ya hemos terminado la pre-configuración del proyecto, y el repositorio remoto se encuentra completamente preparado para comenzar con el desarrollo del proyecto y es por eso que vamos a crear un `tag` para marcar esta fase de pre-configuración del proyecto, y así poder tener un control de versiones más exhaustivo y poder volver a esta fase en caso de ser necesario. Para ello, ejecutamos el siguiente comando:

```
# Creamos un tag para marcar la fase de pre-configuración del proyecto
git tag -a v0.1.0-init -m "Configuración inicial del entorno y documentación"

# Subimos el tag al repositorio remoto
git push origin v0.1.0-init
```

Finalmente, la estructura del entorno pre-configurado del proyecto se muestra a continuación:

```
pcd_entregable1_alfonso/
|-- README.md
|-- .gitignore
|-- docs
|   |-- data
|   |   `-- header_entregable_I.tex
|   |-- images
|   |-- interface
|   |   |-- entregable_I.qmd
|   |   `-- entregable_I.pdf
|   `-- uml
|-- src
`-- tests
```

3.1.6 Subida de los Archivos de Documentación al Repositorio Remoto

Una vez que ya hemos terminado la pre-configuración del proyecto, y el entorno de desarrollo se encuentra completamente preparado para comenzar con el desarrollo del proyecto, vamos a subir los archivos de documentación hasta este apartado al repositorio remoto. Para ello, ejecutamos el siguiente comando:

```
git add docs/data/header_entregable_I.tex docs/interface/entregable_I.pdf docs/interface/entregable_I.qmd

git commit -m "Subiendo los archivos de documentación de la preparación del proyecto"

git push -u origin main
```

3.1.7 Creación de la Rama de Desarrollo

Una vez que ya hemos subido los archivos de documentación de la preparación del proyecto, vamos a crear la rama de desarrollo, la cual se llamará `development`, y en la que se realizará todo el desarrollo del proyecto. Para ello, ejecutamos el siguiente comando:

```
# Creamos y nos desplazamos a la rama de desarrollo
git checkout -b development
```

Ahora, nuestra filosofía de trabajo para toda la documentación se basará en desarrollarla en la rama `main` y que la rama `development` sea una copia de la rama `main` sólo para la documentación, mientras que el desarrollo del código se realizará en la rama `development`. Para ello, siempre que realicemos un cambio en la documentación, lo haremos en la rama `main` y posteriormente fusionaremos los cambios a la rama `development` para mantenerla actualizada con la documentación. Para ello, ejecutamos el siguiente comando cada vez que realicemos un cambio en la documentación:

```
# Sabiendo que estamos en la rama main, realizamos un commit con los cambios en la documentación
git add docs/data/header_entregable_I.tex docs/interface/entregable_I.pdf docs/interface/entregable_I.qmd
git commit -m "Subiendo los archivos de documentación"
git push # main

# Nos desplazamos a la rama de desarrollo
git checkout development

# Fusionamos los cambios de la rama main a la rama de desarrollo
git merge main
git push # development
```

3.2 Diseño del Diagrama de Clases, de Caso de Uso y de Secuencia en UML

Para hablar del diseño del diagrama de clases, de caso de uso y de secuencia en UML, vamos a crear un nuevo apartado para cada uno de ellos, y así poder explicarlos de forma detallada y clara. Cada uno de los diagramas los hemos realizado con código de `mermaid` debido a la incompatibilidad de la herramienta `Umlet` con mi sistema operativo `macOS`, y a que el código de `mermaid` es mucho más fácil de integrar en la documentación, y además es mucho más fácil de modificar y mantener a lo largo del desarrollo del proyecto.

3.2.1 Diagrama de Clases UML

Vamos a comenzar con el diseño del diagrama de clases UML. Para ello, hemos diseñado el diagrama de clases UML del sistema de gestión de mantenimiento de la flota espacial *MiImperio*, el cual se muestra a continuación:

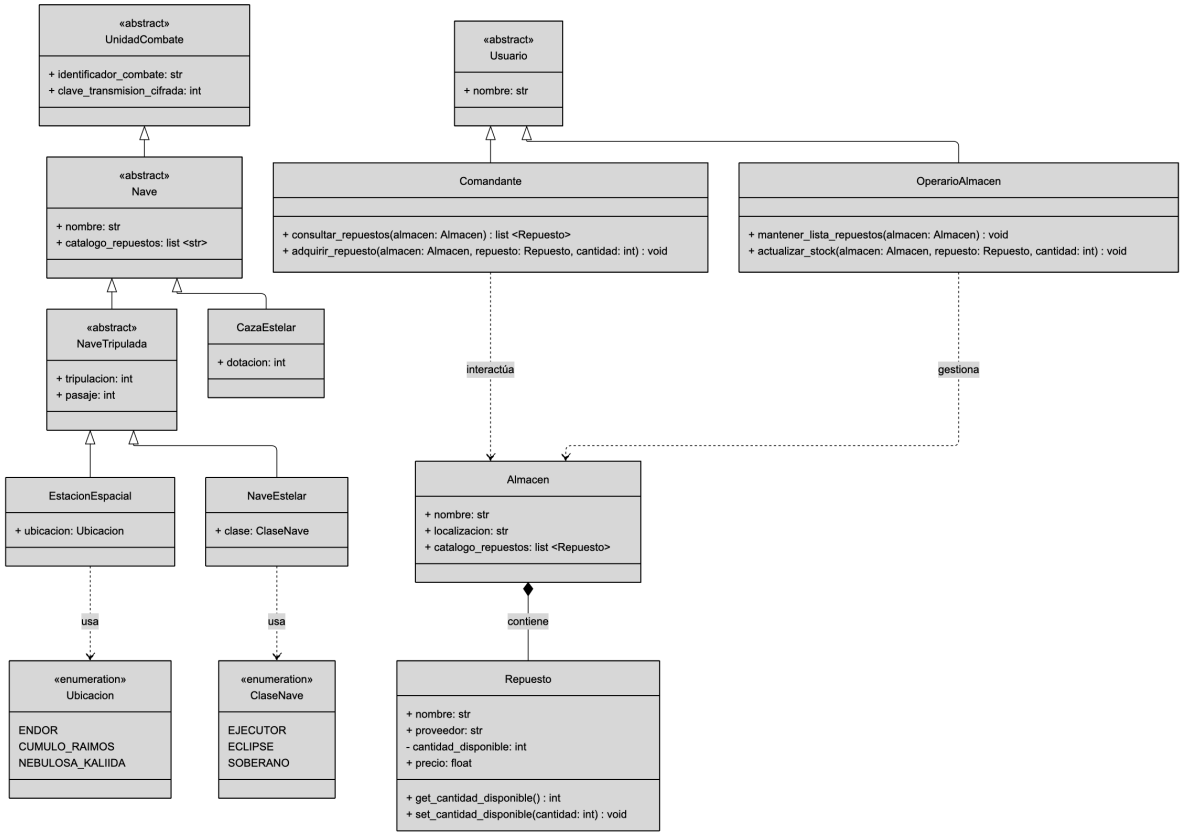

Figure 1: Diagrama de Clases UML del Sistema de Gestión de Mantenimiento de la Flota Espacial.

Podemos descifrar el diagrama de clases UML del sistema *MiImperio* en tres bloques principales, cada uno de ellos con sus propias características y relaciones entre las clases, los cuales vamos a describir a continuación:

1. **Unidades de Combate y Naves Espaciales.** El diseño establece una sólida estructura de herencia para maximizar la reutilización del código:
   - **UnidadCombate (`Clase Abstracta`):** Actúa como la raíz de la jerarquía, garantizando que todo vehículo o nave imperial posea un identificador de combate y una clave cifrada.
   - **Nave (Clase Abstracta):** Hereda de `UnidadCombate` y añade los atributos comunes de toda nave, que son el nombre y el catálogo base de repuestos. De ella descienden directamente los `CazaEstelar`, definiendo su atributo propio de `dotacion`, y la clase abstracta `NaveTripulada`.
   - **NaveTripulada (Clase Abstracta):** Se introduce como una capa intermedia para agrupar los atributos `tripulacion` y `pasaje`, evitando la duplicidad de código. De esta clase heredan las implementaciones concretas de `EstacionEspacial` y `NaveEstelar`.
   - **Enumeraciones (`<<enumeration>>`):** Las propiedades específicas como la `Ubicacion` de las estaciones y la `ClaseNave` de las naves estelares se han modelado mediante tipos enumerados para restringir y validar los valores permitidos según las normativas del Imperio.
2. **Inventario de Almacenes y Repuestos.** Este bloque gestiona los recursos mecánicos del sistema:
   - **Composición:** Existe una relación de composición fuerte con cardinalidad `1..*` entre `Almacen` y `Repuesto`. Es decir, un almacén contiene una o más piezas de repuesto, y cada pieza de repuesto no puede existir sin estar asociada a un almacén.
   - **Encapsulamiento:** En la clase `Repuesto`, el atributo `cantidad_disponible` se ha declarado como privado (marcado con `-`). Para acceder o modificar el stock de forma segura, se han implementado los métodos públicos (getters y setters) como `get_cantidad_disponible()` y `set_cantidad_disponible()`.
3. **Usuarios e Interacciones.** Los actores del sistema se han modelado mediante otra jerarquía de herencia:
   - **Usuario (Clase Abstracta):** Agrupa el atributo común `nombre`.
   - **Comandante y OperarioAlmacen:** Heredan de Usuario y definen los métodos específicos de su rol como `adquirir_repuesto()` o `actualizar_stock()`.
   - **Dependencias:** Se han trazado relaciones de dependencia (flechas discontinuas `..>`) desde los usuarios hacia la clase `Almacen`, indicando que los métodos de los usuarios interactúan y gestionan instancias de los almacenes para operar sobre el inventario.

3.2.2 Diagrama de Casos de Uso UML

Vamos a continuar con el diseño del diagrama de casos de uso UML. Para ello, hemos diseñado el diagrama de casos de uso UML del sistema de gestión de mantenimiento de la flota espacial *MiImperio*, el cual se muestra a continuación:

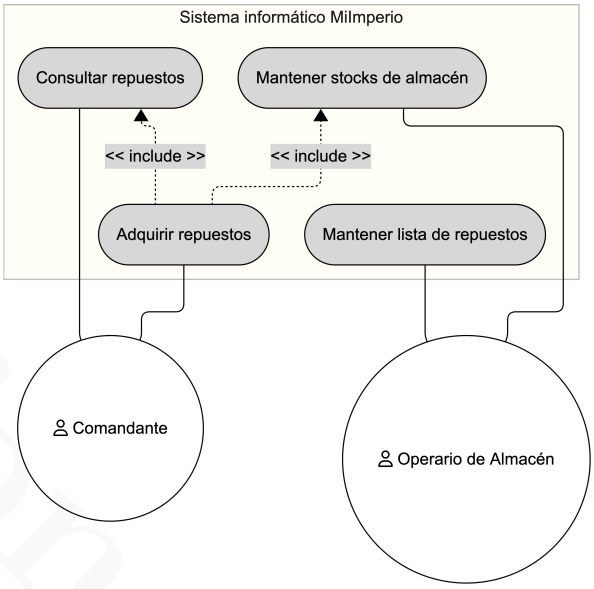

Figure 2: Diagrama de Casos de Uso UML del Sistema de Gestión de Mantenimiento de la Flota Espacial.

En el diagrama de casos de uso definimos los requisitos funcionales del sistema informático *MiImperio*, delimitando claramente las acciones que puede realizar cada perfil de usuario y cómo el sistema responde a dichas peticiones. El modelo se estructura en los siguientes elementos clave:

1. **Frontera del Sistema (System Boundary):** Representada por el contenedor principal "Sistema informático MiImperio", establece el límite lógico del software. Todo lo que ocurre dentro de esta caja es responsabilidad del programa, mientras que los actores que inician las acciones se sitúan en el exterior.
2. **Actores del Sistema:** Se han identificado dos perfiles de usuario principales que interactúan de forma independiente con la plataforma:
   - **Comandante:** Representa a los oficiales al mando de las unidades imperiales. Sus operaciones de negocio consisten en interactuar con los almacenes para `Consultar repuestos` y `Adquirir repuestos` para su flota.
   - **Operario de Almacén:** Es el perfil logístico encargado de la persistencia de datos y el inventario. Sus responsabilidades directas son `Mantener la lista de repuestos` (dar de alta, modificar o eliminar piezas del catálogo) y `Mantener stocks de almacén` (ajustar las cantidades físicas disponibles).
3. **Relaciones de Dependencia (`<<include>>`):** Para evitar la redundancia funcional y garantizar la consistencia de los datos, se ha modelado el caso de uso `Adquirir repuestos` como una operación compleja que desencadena obligatoriamente otros procesos internos. Esto se representa mediante relaciones con el estereotipo `<<include>>`:
   - **Inclusión de Consulta:** Al intentar adquirir un repuesto, el sistema ejecuta implícitamente la acción de `Consultar repuestos` para verificar su existencia en el catálogo y obtener su precio actual.
   - **Inclusión de Actualización de Stock:** Una vez confirmada la adquisición por parte del Comandante, el sistema llama obligatoriamente al proceso de `Mantener stocks de almacén` para descontar de forma automática la cantidad comprada del inventario del almacén, asegurando que el stock sea siempre coherente y en tiempo real.

3.2.3 Diagrama de Secuencia UML

Vamos a finalizar con el diseño del diagrama de secuencia UML. Para ello, hemos diseñado el diagrama de secuencia UML del sistema de gestión de mantenimiento de la flota espacial *MiImperio*, el cual se muestra a continuación:

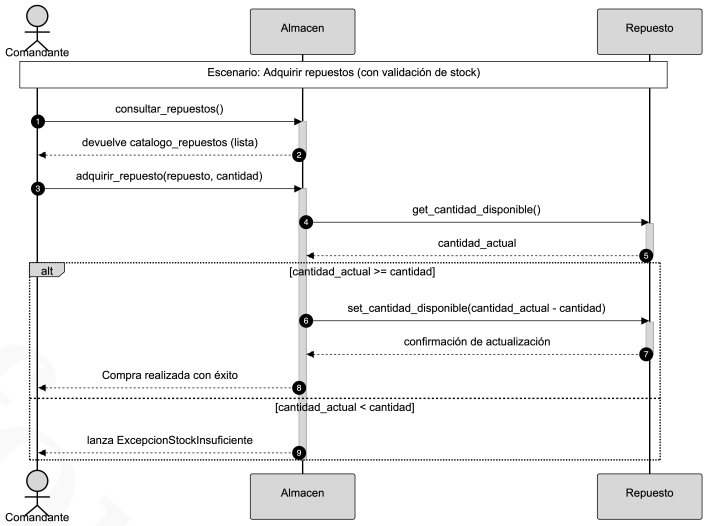

Figure 3: Diagrama de Secuencia UML del Proceso de Adquisición de Repuestos por parte del Comandante.

El diagrama de secuencia modela el comportamiento dinámico del sistema para el escenario de `"Adquirir repuestos"`. En él se detalla cronológicamente el intercambio de mensajes entre los objetos instanciados, prestando especial atención a la validación de la lógica de negocio y la gestión de excepciones. El flujo temporal se divide en las siguientes fases operativas:

1. **Participantes y Líneas de Vida:** El escenario involucra a un actor externo (`Comandante`) interactuando con dos objetos internos del sistema: una instancia de `Almacen` y una instancia específica de `Repuesto`. Los rectángulos de activación sobre las líneas de vida indican el tiempo exacto en el que cada objeto tiene el control de la ejecución.

2. **Fase de Consulta Inicial (Pasos 1-2):** Antes de realizar una compra, el Comandante solicita ver las piezas disponibles mediante el mensaje `consultar_repuestos()`. El objeto `Almacen` procesa la petición y devuelve el catálogo completo de repuestos, permitiendo al usuario decidir qué pieza necesita y en qué cantidad.

3. **Inicio de la Transacción y Validación (Pasos 3-5):** El Comandante inicia la transacción enviando el mensaje `adquirir_repuesto(repuesto, cantidad)`. En este punto, el sistema demuestra un diseño fuertemente encapsulado: el `Almacen` no accede directamente al stock de la pieza, sino que le envía el mensaje `get_cantidad_disponible()` al objeto `Repuesto`. El repuesto responde entregando su `cantidad_actual`.

4. **Toma de Decisiones y Gestión de Excepciones (Pasos 6-9):** Una vez conocida la cantidad física disponible, el flujo entra en un fragmento combinado de tipo `alt` (alternativa), que modela la lógica condicional (`if/else`) de la compra:

   - **Escenario de Éxito (`cantidad_actual >= cantidad`):** Si hay stock suficiente, el `Almacen` envía el mensaje `set_cantidad_disponible()` al `Repuesto`, calculando y pasándole el nuevo stock restante. Tras la confirmación interna, el almacén notifica al Comandante que la compra se ha realizado con éxito.

   - **Escenario de Fallo (`cantidad_actual < cantidad`):** Si el Comandante solicita más unidades de las que físicamente existen, el sistema aborta la operación matemática. El `Almacen` interrumpe el flujo normal y lanza una excepción personalizada (`ExcepcionStockInsuficiente`) hacia el Comandante, garantizando que el inventario nunca adquiera valores negativos y cumpliendo con los requisitos de robustez del software.

3.2.4 Publicación de los Diagramas UML en el Repositorio Remoto

Ahora que ya hemos diseñado los diagramas de clases, de caso de uso y de secuencia en UML, vamos a subirlos al repositorio remoto para tenerlos disponibles y poder consultarlos a lo largo del desarrollo del proyecto. Para ello, ejecutamos los siguientes comandos:

```
# Confirmamos que estamos en la rama main para subir los diagramas UML a la rama principal
git checkout main

# Añadimos los archivos de la documentación de los diagramas UML
git add docs/data/header_entregable_I.tex docs/interface/entregable_I.pdf docs/interface/entregable_I.qmd

# Añadimos las imágenes de los diagramas UML
git add docs/images/diagrama_clases.png docs/images/diagrama_casos_uso.png docs/images/diagrama_secuencia.
     png

# Añadimos los archivos de código de los diagramas UML
git add docs/uml/diagrama_clases.mmd docs/uml/diagrama_casos_uso.mmd docs/uml/diagrama_secuencia.mmd

# Realizamos un commit para subir los diagramas UML al repositorio remoto
git commit -m "Subiendo los diagramas UML del sistema de gestión de mantenimiento de la flota espacial"

# Subimos los cambios a la rama main
git push # main
```

3.2.5 Creación del Tag de Diagramas UML

Una vez que ya hemos subido los diagramas UML al repositorio remoto, vamos a crear un `tag` en la rama `main` para marcar esta fase de diseño del proyecto, y así poder tener un control de versiones más exhaustivo y poder volver a esta fase en caso de ser necesario. Para ello, ejecutamos el siguiente comando:

```
# Creamos un tag para marcar la fase de diseño del proyecto
git tag -a v0.2.0-design -m "Diseño de los diagramas UML del sistema de gestión de mantenimiento de la flota
      espacial"

# Subimos el tag al repositorio remoto
git push origin v0.2.0-design
```

3.3 Implementación en Python del Sistema de Gestión de la Flota Espacial

3.3.1 Implementación del Código Fuente

Comenzaremos la implementación del sistema diseñando la jerarquía de carpetas para el entorno de desarrollo. Todo el código de la aplicación se centralizará en el directorio `src`, el cual contendrá los siguientes archivos clave para el proyecto:

- **El archivo `main.py`:** Actuará como el punto de entrada principal del programa. En él se realizará la instanciación de las diferentes clases del sistema para demostrar su correcto funcionamiento y planificar el flujo de ejecución.
- **El archivo `flota.py`:** Constituye el núcleo del modelo. Contendrá la implementación de toda la jerarquía de clases relacionadas con la flota espacial, incluyendo las enumeraciones, las naves y las unidades de combate.
- **El archivo `__init__.py`:** Este archivo se incluye para que Python reconozca la carpeta `src` como un paquete modular, lo que nos permitirá importar las clases y funciones definidas en nuestro código fuente de forma limpia y estructurada.

La configuración final del directorio para el código fuente la vemos continuación:

```
src/
|-- __init__.py          # Archivo para convertir la carpeta src en un paquete de Python
|-- main.py              # Punto de entrada del programa
`-- flota.py             # Implementación de las clases relacionadas con la flota espacial
```

Uso Andrés

3.3.2 Ejecución del Código Fuente

Como el código fuente lo hemos documentado y le hemos añadido anotaciones en sus propios archivos `flota.py` y `main.py`, vamos a inicializar el sistema de gestión mecánica y ejecutar la prueba del sistema, basta con situarse en la `raíz` del proyecto y ejecutar el archivo principal mediante el intérprete de Python:

```
# Desde la raíz del proyecto (pcd_entregable1_alfonso)
python3 src/main.py
```

Y, obtendríamos la siguiente salida en la terminal:

```
==================================================
SISTEMA DE GESTIÓN MECÁNICA - MiImperio
==================================================

[1] INSTANCIANDO LA FLOTA ESPACIAL...
    -> Creada Estación: Estrella de la Muerte en Endor
    -> Creada Nave Estelar: Ejecutor Supremo clase Ejecutor
    -> Creado Caza: Caza TIE Alfa con dotación de 1

[2] CONFIGURANDO ALMACENES Y PERSONAL...
    -> Almacén 'Almacén Imperial Central' ubicado en Coruscant listo para operar.
    -> Operario 'TK-421' listo para gestionar el inventario.
    -> Comandante 'Darth Vader' listo para operar.

[3] OPERARIO ACTUALIZANDO INVENTARIO...
    -> Repuesto 'Motor Iónico TIE' añadido a Almacén Imperial Central con stock inicial de 50 unidades.
    -> Repuesto 'Panel Solar TIE' añadido a Almacén Imperial Central con stock inicial de 10 unidades.

[4] COMANDANTE ADQUIRIENDO REPUESTOS...
    -> Intentando comprar 20 unidades de 'Motor Iónico TIE'...
        Éxito: Se han adquirido 20 unidades de 'Motor Iónico TIE'. Disponibles restantes: 30.
    -> Intentando comprar 20 unidades de 'Panel Solar TIE'...
        Fracaso: Stock insuficiente para 'Panel Solar TIE'. Solicitados: 20, Disponibles: 10.

==================================================
FIN DE LA SIMULACIÓN
==================================================
```

También, podemos ver este resultado de la terminal en la siguiente captura de pantalla:

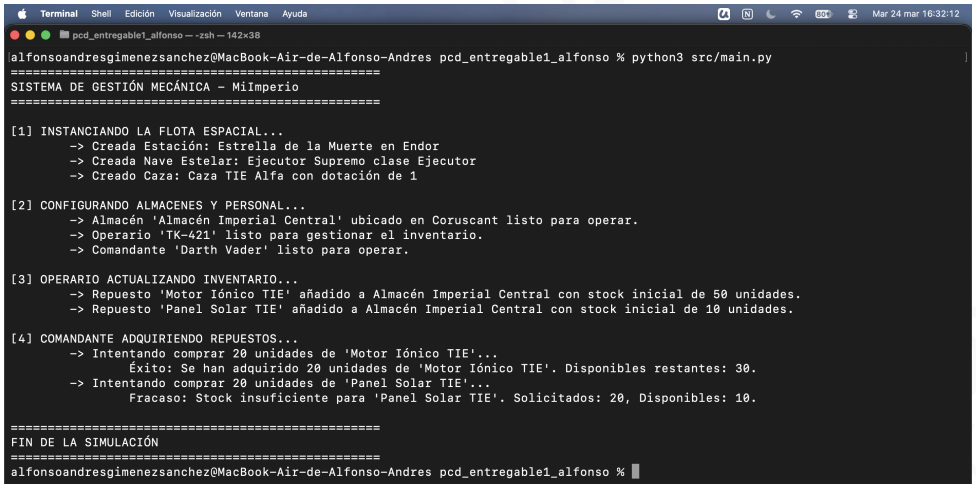

Figure 4: Salida en la Terminal tras Ejecutar el Archivo Principal del Proyecto.

3.3.3 Creación del Tag de Implementación del Código Fuente

Una vez que ya hemos implementado el código fuente del sistema de gestión de mantenimiento de la flota espacial, vamos a publicarlo en nuestro repositorio remoto y a crear un `tag` en la rama `development` para marcar esta fase de implementación del proyecto. Para ello, ejecutamos los siguientes comandos:

```
# Confirmamos que estamos en la rama de desarrollo para subir el código fuente a la rama de desarrollo
git checkout development

# Añadimos los archivos de código fuente
git add src/__init__.py src/flota.py src/main.py

# Realizamos un commit para subir el código fuente al repositorio remoto
git commit -m "Implementación del código fuente del sistema de la flota espacial"

# Subimos los cambios a la rama development
git push # development

# Creamos un tag para marcar la fase de implementación del código fuente
git tag -a v0.3.0-implementation -m "Implementación del código fuente del sistema de la flota espacial"

# Subimos el tag al repositorio remoto
git push origin v0.3.0-implementation
```

Finalmente, publicamos la documentación de la implementación del código fuente en el repositorio remoto en la rama `main` para que esté disponible para su consulta, y posteriormente fusionamos los cambios a la rama de desarrollo para mantenerla actualizada con la documentación. Para ello, ejecutamos los siguientes comandos:

```
# Confirmamos que estamos en la rama main para subir la documentación
git checkout main

# Añadimos los archivos de la documentación de la implementación del código fuente
git add docs/data/header_entregable_I.tex docs/interface/entregable_I.pdf docs/interface/entregable_I.qmd
     docs/images/salida_terminal.png

# Realizamos un commit para subir la documentación del código fuente al repositorio remoto
git commit -m "Subiendo la documentación de la implementación del código fuente del sistema de gestión de
     mantenimiento de la flota espacial"

# Subimos los cambios a la rama main
git push # main

# Nos desplazamos a la rama de desarrollo
git checkout development

# Fusionamos los cambios de la rama main a la rama de desarrollo
git merge main
git push # development
```

3.4 Realización de Pruebas del Código Fuente con `pytest`

3.4.1 Implementación de Tests con `pytest`

Para garantizar la robustez y fiabilidad del sistema de gestión de mantenimiento de la flota espacial, hemos implementado un conjunto de tests unitarios utilizando el framework `pytest`. Todo el código relacionado con las pruebas se ha centralizado en el directorio `tests`, el cual contiene los siguientes archivos clave para el proyecto:

- **El archivo `test_flota.py`:** Contiene los tests unitarios específicos para validar la correcta implementación de la flota espacial, incluyendo la creación de naves, la gestión de repuestos y la interacción entre los diferentes componentes del sistema.
- **El archivo `__init__.py`:** Este archivo se incluye para que Python reconozca la carpeta `tests` como un paquete modular, lo que nos permitirá importar las funciones y clases definidas en nuestros tests de forma limpia y estructurada.

La configuración final del directorio para los tests unitarios la vemos a continuación:

```
tests/
|-- __init__.py          # Archivo para convertir la carpeta tests en un paquete de Python
`-- test_flota.py        # Tests unitarios para la implementación de la flota espacial
```

3.4.2 Ejecución de los Tests con `pytest`

Para ejecutar los tests unitarios implementados con `pytest` vamos a situarnos en la raíz del proyecto y ejecutamos el siguiente comando en la terminal:

```
# Desde la raíz del proyecto (pcd_entregable1_alfonso)
pytest tests/test_flota.py
```

Y, obtendríamos la siguiente salida en la terminal:

```
=================================================== test session starts
     ===================================================
platform darwin -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0
Fugue tests will be initialized with options:
rootdir: /Users/alfonsoandresgimenezsanchez/Desktop/Estudios/Universidad/Cursos/2_Curso/segundo_cuatrimestre
     /programacion_para_ciencia_de_datos/2025_2026/practicas/entregable_I/pcd_entregable1_alfonso
plugins: anyio-4.11.0, html-4.0.2, xdist-3.8.0, timeout-2.4.0, metadata-3.1.1, fugue-0.9.1, seleniumbase
     -4.44.20, ordering-0.6, typeguard-4.4.4, hypothesis-6.142.4, Faker-37.12.0, dash-2.18.2, rerunfailures
     -16.1, cov-7.0.0
collected 4 items

tests/test_flota.py ....
            [100%]

==================================================== 4 passed in 0.21s
     ===================================================
```

También, podemos ver este resultado de la terminal en la siguiente captura de pantalla:

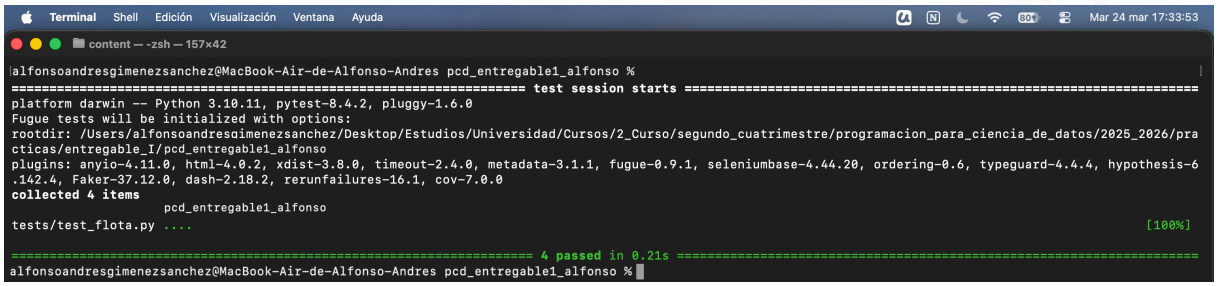

Figure 5: Salida en la Terminal tras Ejecutar los Tests Unitarios con `pytest`.

3.4.3 Creación del Tag de Pruebas Unitarias

Una vez que ya hemos implementado los tests unitarios y los hemos ejecutado con éxito, vamos a publicar en el repositorio remoto los archivos correspondientes a los tests unitarios y a crear un último `tag` en la rama `development` para marcar esta fase de pruebas del proyecto. Para ello, ejecutamos los siguientes comandos:

```
# Confirmamos que estamos en la rama de desarrollo para crear el tag de pruebas
git checkout development

# Añadimos los archivos de código de los tests unitarios
git add tests/__init__.py tests/test_flota.py

# Realizamos un commit para subir los tests unitarios al repositorio remoto
git commit -m "Implementación de tests unitarios con pytest para el sistema de gestión de mantenimiento de
     la flota espacial"

# Subimos los cambios a la rama development
git push # development

# Creamos un tag para marcar la fase de pruebas del proyecto
git tag -a v0.4.0-testing -m "Implementación de tests unitarios con pytest para el sistema de gestión de
     mantenimiento de la flota espacial"

# Subimos el tag al repositorio remoto
git push origin v0.4.0-testing
```

Después de crear el tag vamos a actualizar en el repositorio remoto la documentación en la rama `main` para incluir la sección de pruebas unitarias, y posteriormente fusionamos los cambios a la rama de desarrollo para mantenerla actualizada con la documentación. Para ello, ejecutamos los siguientes comandos:

```
# Confirmamos que estamos en la rama main para subir la documentación de las pruebas unitarias
git checkout main

# Añadimos los archivos de la documentación de las pruebas unitarias
git add docs/data/header_entregable_I.tex docs/interface/entregable_I.pdf docs/interface/entregable_I.qmd
     docs/images/salida_tests.png

# Realizamos un commit para subir la documentación de las pruebas unitarias al repositorio remoto
git commit -m "Subiendo la documentación de las pruebas unitarias del sistema de gestión de mantenimiento de
      la flota espacial"

# Subimos los cambios a la rama main
git push # main

# Nos desplazamos a la rama de desarrollo
git checkout development

# Fusionamos los cambios de la rama main a la rama de desarrollo
git merge main
git push # development
```

3.5 Cambios Finales y Cierre del Proyecto

Llegados a este punto, ya hemos completado todas las fases del proyecto; desde la preparación del entorno de trabajo, pasando por el diseño de los diagramas UML, hasta la implementación del código fuente y la realización de pruebas unitarias.

Como paso final, vamos a actualizar el archivo `README.md` para incluir un resumen del proyecto, la estructura del repositorio y las instrucciones de ejecución. Una vez hecho esto, realizaremos la fusión definitiva basada en la integración de todo el código fuente y los tests de la rama de desarrollo hacia la rama principal.

Para llevar a cabo esta maniobra final, ejecutamos los siguientes comandos:

```
# Nos situamos en la rama principal y preparamos el README final
git checkout main
git add README.md
git commit -m "Actualización del README.md con instrucciones y estructura final"

# Traemos todo el código y tests desde development hacia main
git merge development
git push # main
```

Finalmente, con la rama `main` conteniendo la versión absoluta y completa del proyecto (código y documentación), creamos un último `tag` para marcar la finalización. Esto nos permitirá generar posteriormente una *Release* oficial en GitHub. Ejecutamos los siguientes comandos:

```
# Creamos la etiqueta de la versión final y la publicamos en el repositorio remoto
git tag -a v1.0.0-final -m "Versión final del proyecto de gestión de mantenimiento de la flota espacial"

# Subimos el tag al repositorio remoto
git push origin v1.0.0-final
```

Llegado a estas alturas, damos por finalizado este proyecto de gestión de mantenimiento de la flota espacial.